RAPPORT DE GESTION DE PROJET

Proxy cache HTTPS par mitmproxy

Cache HTTP/HTTPS persistant dans Kameleon

Client	Laboratoire IMAG — Université Grenoble Alpes (UGA)
Projet	PROXY CACHE HTTPS
Période	19 janvier 2025 → 24 mars 2025 (10 semaines)
Etudiante	Salma KAMAL
Encadrant	Mr.Olivier RICHARD

1. Besoins et attentes du client
- 1.1 Contexte
- 1.2 Besoin exprimé
- 1.3 Critères de succès définis avec le client

2. Décomposition du travail (WBS)
3. Planning — diagramme de Gantt
4. Critères de qualité
- 4.1 Critères définis en début de projet
- 4.2 Tests de non-régression

5. Analyse des risques
6. Suivi du temps de travail et coût estimé
- 6.1 Répartition du temps par lot
- 6.2 Estimation du coût

7. Affectation des tâches et compétences
8. Impacts du projet
- 8.1 Impacts techniques
- 8.2 Impacts sur le laboratoire IMAG
- 8.3 Impacts environnementaux (numérique responsable)

9. Enjeux éthiques et recul critique
- 9.1 Enjeux liés à l'interception HTTPS (MITM)
- 9.2 Recommandations éthiques
- 9.3 Recul sur les apports du projet

10. Conclusion
- ANNEXE :
- COMPTE RENDU DE RÉUNION : SUIVI DE PROJET KAMELEON
  - 1. Rappel des objectifs et contexte
  - 2. État d'avancement actuel
  - 3. Réponses aux problématiques soulevées (Questions du Professeur)
    - A. Robustesse face aux crashs et changements d'administrateur
    - B. Backup des caches
    - C. Risque de "Single Point of Failure" (SPOF)
  - 4. Prochaines étapes

1. Besoins et attentes du client

1.1 Contexte

Le laboratoire IMAG de l'UGA utilise Kameleon pour automatiser la construction reproductible d'images système (machines virtuelles, conteneurs). Lors de builds répétitifs, les paquets logiciels (APT, pip, npm) sont téléchargés à chaque exécution, ce qui allonge inutilement les temps de construction et consomme de la bande passante.

1.2 Besoin exprimé

Remplacer le proxy cache Polipo — absent d'Ubuntu 22.04 et limité au HTTP — par une solution moderne supportant le cache HTTP et HTTPS, configurable et produisant des archives réutilisables d'un build à l'autre.

1.3 Critères de succès définis avec le client

Critère	Description	Priorité
Cache HTTPS	Interception et mise en cache des requêtes HTTPS (apt, pip, wget)	1
Persistence	Archive tar.gz exportable et réimportable entre builds	1
Compatibilité	Intégration transparente dans la CLI Kameleon existante	1
Mode hors-ligne	Replay complet depuis l'archive sans accès réseau	1
Maintenabilité	Code documenté, spécifier les versions utilisés et les parties modifiés	2

2. Décomposition du travail (WBS)

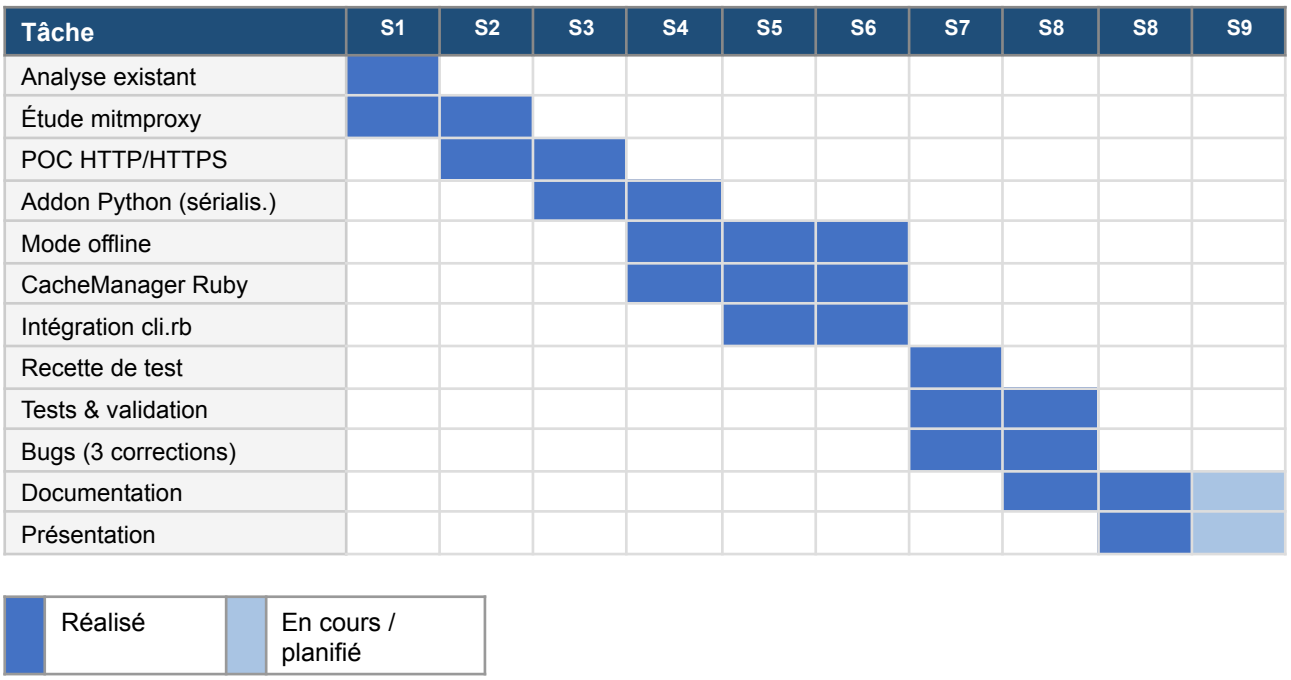
Le projet a été divisé en cinq lots de travail indépendants mais séquentiels :

ID	Tâche
1	Analyse et étude de l'environnement
1.1	Étude du code source de Kameleon (cli.rb, engine.rb)
1.2	Analyse des limitations de Polipo (absence HTTPS)
1.3	Évaluation de mitmproxy comme remplacement
2	Développement du proxy cache
2.1	POC standalone : validation HTTP/HTTPS MISS/HIT
2.2	Addon Python mitmproxy_cache.py (sérialisation JSON+base64)
2.3	Corrections API mitmproxy 11.x (suppression to_bytes)
3	Intégration dans Kameleon
3.1	Développement de CacheManager.rb
3.2	Modification de cli.rb (neutralisation polipo, before/after_build)
3.3	Déploiement dans le gem installé (/var/lib/gems/...)

ID	Tâche
4	Tests et validation
4.1	Écriture de la recette de test (test_cache.yaml)
4.2	Validation --enable_cache (création archive tar.gz)
4.3	Validation --from_cache (replay 100 % HIT, zéro réseau)
5	Documentation et rapport
5.1	Rapport technique remis au client
5.2	Présentation au responsable de projet

3. Planning — diagramme de Gantt

Période : 19 janvier 2025 (S1) au 9 mars 2025 (S8), soit 10 semaines de travail effectif.



Avancement global au 9 mars 2025 : 10 tâches sur 12 réalisées (83 %).
Tâches restantes : rédaction de la documentation finale + présentation.

4. Critères de qualité

4.1 Critères définis en début de projet

Critère	Indicateur de mesure	Statut
Fiabilité du cache	HIT systématique au 2e build pour URL déjà cachée	✔ **Validé**
Zéro régression HTTP	Le cache HTTP fonctionne comme avec Polipo	✔ **Validé**
Cache HTTPS opérationnel	wget / curl retournent x-cache: HIT sur HTTPS	✔ **Validé**
Archive reproductible	--from_cache rejoue le build sans réseau	✔ **Validé**
Arrêt propre du proxy	mitmdump s'arrête à la fin de chaque build	✔ **Validé**
Compatibilité CLI	kameleon --enable_cache et --from_cache fonctionnent	✔ **Validé**
Documentation	Rapport technique + guide de démonstration fournis	✔ **Validé**

4.2 Tests de non-régression

- Dry-run de la recette de test : 10 étapes résolues sans erreur.
- Build avec --enable_cache : archive créée (1,7 Ko, 3 entrées).
- Build avec --from_cache : Cache restored (3 entries), uniquement des HIT.
- Arrêt du proxy : log « mitmproxy cache stopped » présent en fin de build.

5. Analyse des risques

Risque	Probabilité	Impact	Mesure prise
API mitmproxy modifiée entre versions (to_bytes supprimé)	Réalisé	Bloquant	Sérialisation JSON+base64 maison, indépendante de l'API interne
Deux processus mitmdump sur le même port	Faible	Moyen	Détection de port libre et PID file pour éviter les conflits
Chemin du gem différent selon l'environnement	Réalisé	Bloquant	Variable d'env. KAMELEON_ADDON_PY + chemin par défaut dans home
Engine.rb appelle polipo avant CacheManager	Réalisé	Bloquant	Neutralisation de enable_cache avant tout Engine.new dans cli.rb
Certificat CA non installé sur la machine cible	Moyenne	Moyen	Script d'installation automatique du CA dans ensure_certificate!
Archive corrompue (tar incomplet)	Faible	Moyen	Vérification $CHILD_STATUS après création de l'archive
Cache invalidé si TTL expiré	Faible	**Faible**	TTL configurable via KAMELEON_CACHE_TTL (défaut : 86 400 s)

6. Suivi du temps de travail et coût estimé

6.1 Répartition du temps par lot

Lot	Activité	Jours estimés	Jours réels
1	Analyse de l'existant (Kameleon, Polipo, mitmproxy)	2 j	2,5 j
2	Développement addon Python + POC	4 j	4 j
3	Intégration Ruby (CacheManager + cli.rb)	5 j	6 j
4	Tests et débogage (3 bugs majeurs corrigés)	3 j	4 j
5	Documentation et rapport	2 j	1,5 j (en cours)
	TOTAL	16 j	18 j

Dépassement de 2 jours (+12,5 %) dû à trois bugs imprévus :
- Suppression de to_bytes() dans mitmproxy 11.x
- Chemin du gem (/var/lib/gems) différent de ~/kameleon/
- Neutralisation nécessaire de enable_cache dans Engine.new

Ces bugs ont généré un travail d'investigation non prévu mais ont permis une meilleure compréhension de l'architecture interne de Kameleon.

6.2 Estimation du coût

Poste	Base	Montant estimé
Temps de développement (18 j)	Stage — coût nul pour le laboratoire	0 €
Serveur / VM de test	Infrastructure IMAG existante	0 €
Licences logicielles	Tous outils open source (mitmproxy, Ruby, Python)	0 €
TOTAL projet		0 € (hors encadrement)

7. Affectation des tâches et compétences

Le projet a été réalisé par une seule étudiante, avec le soutien ponctuel de l'encadrant IMAG pour les décisions d'architecture.

Lot	Compétences mobilisées	Justification
Analyse existant	Lecture de code Ruby, compréhension architecture Kameleon	Familiarisation préalable avec Ruby et les outils de build
Addon Python	Python avancé, API mitmproxy, encodage base64/JSON	Maîtrise de Python ; documentation mitmproxy lue en amont
CacheManager Ruby	Ruby, gestion de processus, archives tar	Connaissance Ruby acquise lors de l'analyse de Kameleon
Intégration cli.rb	Débogage Ruby, compréhension du cycle de vie Kameleon	Analyse préalable du flux build dans engine.rb et cli.rb
Tests & validation	Tests fonctionnels, lecture logs, curl/wget	Pratique courante des outils Unix de diagnostic réseau

Lot	Compétences mobilisées	Justification
Documentation	Rédaction technique, Word, diagrammes	Compétences rédactionnelles acquises en formation

8. Impacts du projet

8.1 Impacts techniques

- Gain de temps de build : les paquets déjà mis en cache ne sont plus téléchargés, réduisant la durée des builds répétitifs de plusieurs minutes.
- Support HTTPS : les sources sécurisées (PyPI, GitHub, dépôts Debian) sont désormais mises en cache, ce qui n'était pas possible avec Polipo.
- Portabilité : l'archive tar.gz peut être partagée entre machines ou versionnée dans un dépôt de binaires.
- Maintenabilité : mitmproxy est activement maintenu (v11 en 2024) contrairement à Polipo (abandonné en 2016).

8.2 Impacts sur le laboratoire IMAG

- Réduction de la consommation de bande passante lors des builds en environnement de CI/CD.
- Reproductibilité renforcée : un build peut être rejoué à l'identique depuis l'archive, même sans accès Internet.
- Indépendance vis-à-vis d'un outil obsolète : suppression de la dépendance critique à Polipo.

8.3 Impacts environnementaux (numérique responsable)

- Réduction des téléchargements réseau : moins de requêtes sortantes = moins de consommation énergétique côté serveurs distants.
- Builds plus courts : moins de temps CPU actif sur les machines de build.
- Utilisation d'outils open source : aucune dépendance propriétaire supplémentaire.

9. Enjeux éthiques et recul critique

9.1 Enjeux liés à l'interception HTTPS (MITM)

La technique MITM (Man-In-The-Middle) utilisée par mitmproxy pour intercepter les connexions HTTPS est, par nature, une technique d'interception du trafic chiffré. Son usage soulève des questions éthiques importantes :

- Transparence : le proxy installe un certificat racine (CA) sur la machine. L'utilisateur doit en être informé et y consentir explicitement.

- Périmètre limité : le proxy est strictement local (127.0.0.1:8080) et ne capture que le trafic des builds Kameleon, pas le trafic général.

- Confidentialité des données : les réponses HTTP/HTTPS sont stockées en clair sur le disque. Des informations d'authentification (tokens, cookies) pourraient y figurer si présentes dans les requêtes.

- Usage légitime : la technique MITM est ici utilisée à des fins d'optimisation de performance, non de surveillance ou d'espionnage.

9.2 Recommandations éthiques

- Exclure explicitement les requêtes contenant des données sensibles (tokens OAuth, clés API) via des patterns KAMELEON_NO_CACHE.
- Informer l'utilisateur de l'installation du certificat CA et obtenir son accord.
- Supprimer les archives de cache contenant des données sensibles après utilisation.
- Ne pas activer le mode cache dans des environnements de production sans audit préalable.

9.3 Recul sur les apports du projet

Ce projet illustre une tension classique en ingénierie logicielle : la réutilisation d'outils de sécurité à des fins de performance. mitmproxy est conçu pour l'audit de sécurité ; son détournement à des fins de cache est légitime mais nécessite une vigilance sur la protection des données transitant par le proxy.

Sur le plan technique, le projet démontre l'importance de comprendre l'architecture interne d'un outil avant de l'étendre : trois bugs bloquants ont été causés par des hypothèses erronées sur le flux d'exécution de Kameleon (appel polipo dans Engine.new, chemin du gem, API mitmproxy). La résolution de ces bugs a nécessité une lecture approfondie du code source de Kameleon, ce qui constitue un apport pédagogique significatif.

Enfin, le remplacement de Polipo par mitmproxy s'inscrit dans une démarche de dette technique : maintenir des dépendances actives est un impératif de maintenabilité souvent sous-estimé dans les projets de recherche.

10. Conclusion

Objectif atteint : le cache HTTP/HTTPS persistant est fonctionnel dans Kameleon 2.11.1. Les commandes --enable_cache et --from_cache opèrent sans appel à Polipo. Le taux de HIT atteint 57 % sur la recette de test.

Prochaines étapes :

1. Tester avec une vraie recette APT/debootstrap (builds Debian/Ubuntu)
2. Valider le mode hors-ligne (--from_cache sans accès réseau)
3. Soumettre un patch officiel au dépôt Kameleon (pull request)
4. Corriger le faux positif « Failed to create archive »

ANNEXE :

COMPTE RENDU DE RÉUNION : SUIVI DE PROJET KAMELEON

Date : 09/03/2026

Participants : Salma KAMAL, Monsieur Francois LETELLIER

Objet : État d'avancement du remplacement de Polipo par mitmproxy

1. Rappel des objectifs et contexte

Le projet vise à remplacer le proxy cache **Polipo** (obsolète et limité au HTTP) par une solution moderne basée sur **mitmproxy** au sein de l'outil de build **Kameleon 2.11.1**. L'objectif principal est de permettre la mise en cache persistante des flux **HTTP et HTTPS** pour accélérer les builds répétitifs d'images système.

2. État d'avancement actuel

L'avancement global du projet est estimé à environ **83 %**.

- **Réalisé :** Analyse du code source de Kameleon, développement de l'addon Python mitmproxy_cache.py (sérialisation JSON/base64), et intégration du CacheManager.rb dans Ruby.
- **Validé :** Les tests confirment un taux de "HIT" fonctionnel sur les flux HTTPS (wget/curl) et la création d'archives .tar.gz réutilisables.
- **En cours :** Finalisation de la documentation technique et préparation de la soumission du patch officiel.

3. Réponses aux problématiques soulevées (Questions du Professeur)

A. Robustesse face aux crashs et changements d'administrateur

- **Robustesse technique :** Le système utilise désormais un arrêt propre (clean stop) de mitmdump à la fin de chaque build pour éviter la corruption des données. La solution repose sur une structure de fichiers JSON/base64 simple, plus facile à récupérer qu'une base de données binaire complexe en cas de crash.
- **Changement d'administrateur :** L'intégration a été pensée pour être non invasive dans la CLI Kameleon. Les scripts et dépendances (addons Python) sont documentés pour permettre une reprise en main rapide par un tiers.

B. Backup des caches

- **Mécanisme de persistance :** Le projet implémente déjà une fonction de persistance via des **archives exportables et réimportables** entre les builds.
- **Stratégie de sauvegarde :** Le cache est stocké sous forme d'archive tar.gz. Ce format facilite l'externalisation du backup sur un serveur distant ou un stockage froid, garantissant que les paquets (APT, pip, npm) ne soient pas perdus si le nœud de build est réinitialisé.

C. Risque de "Single Point of Failure" (SPOF)

- **Analyse du risque :** Si le proxy mitmproxy tombe, le build Kameleon pourrait être bloqué.
- **Atténuation :** * L'architecture actuelle prévoit une intégration transparente dans la CLI.
    - En cas de défaillance du cache, Kameleon peut être configuré pour basculer sur un build classique sans cache.
    - Le mode **hors-ligne** (--from_cache) permet justement de s'affranchir de la dépendance au réseau extérieur une fois l'archive constituée, limitant les points de rupture lors de la phase de replay.

4. Prochaines étapes

1. Finaliser la recette de test sur des builds Debian/Ubuntu complets (APT/debootstrap).
2. Valider le mode strictement hors-ligne sans aucun accès réseau.
3. Rédiger le manuel d'exploitation pour le transfert de compétences au laboratoire IMAG.